

# USO DE REDES NEURAIS PARA APROXIMAR OS PESOS DE INTERPOLAÇÃO DO SISTEMA HIG-FLOW

15 de maio de 2026

## 1 Introdução

Simular fluidos em malhas não estruturadas (irregulares) é um desafio, tanto em questão de modelagem, quanto na capacidade computacional necessária. Nesses casos, normalmente é necessário realizar interpolações geométricas, operações que são altamente dependentes das células ao redor do ponto desejado, o que impacta negativamente na complexidade dessa operação, principalmente em três dimensões. Com isso em mente, o trabalho de [1] propõe um método de diferenças finitas para malhas hierárquicas usando MLS (*Moving Least Squares*), um método *meshless*, para realizar as interpolações.

No *HiG-Flow*, software que implementa as ideias de [1] na linguagem de programação C, a malha hierárquica é armazenada em uma estrutura de dados chamada *HiG-Tree* (*Hierarchical Grid Tree*), uma árvore em que cada folha (célula) pode ser refinada geometricamente em subcélulas, dando origem a novas folhas, como pode ser visto na figura 1.

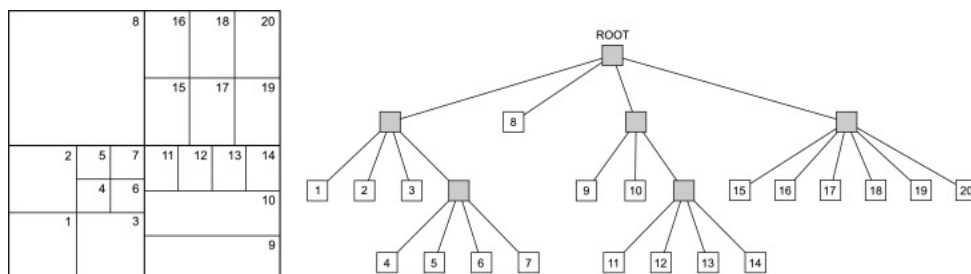


Figura 1: Exemplo da estrutura de dados higtree  
Fonte: [1]

Note que, para usar as diferenças finitas clássicas, os pontos da malha devem estar organizados em retângulos (no caso 2D), o que nem sempre ocorre com uma malha

hierárquica, tornando necessário as interpolações para pontos fora da malha. O método proposto por [1] é custoso computacionalmente, ao passo que para cada interpolação, um sistema deve ser resolvido. Isso não é um grande problema em malhas estáticas, já que o cálculo dessas interpolações pode ser feito como um passo de pré-processamento e os pesos de interpolação podem ser guardados em *cache*.

O desafio fica claro quando mudamos para uma malha dinâmica. Para cada passo de tempo da simulação, diversas interpolações devem ser calculadas, o que pode se tornar um gargalo em uma malha com milhões de células.

Atualmente, o sistema *HiG-Flow* implementa a técnica *PLIC-VOF* (Piecewise Linear Interface Calculation - Volume of Fluid) para simular escoamentos bifásicos [2]. Além disso, estão sendo desenvolvidas ferramentas para o refinamento adaptativo da malha, principalmente ao redor das interfaces entre os fluidos. Nesses locais do domínio, é necessário uma malha mais fina, para evitar o *smearing* (espalhamento) da interface e limitar melhor a fração de volume  $f$  para  $f \in [0, 1]$  [3], [4].

Com isso em mente, para tentar reduzir o gargalo computacional associado às interpolações, uma saída é utilizar redes neurais para inferir os pesos de interpolação da malha. No entanto, note que, para cada ponto onde os pesos devem ser calculados, há um valor variável de células adjacentes utilizadas.

## 2 Objetivos

O presente projeto tem como objetivo principal investigar, desenvolver e avaliar o uso de Redes Neurais Artificiais (RNAs), como um método eficiente e robusto para calcular os pesos de interpolação espacial no sistema de simulação de fluidos *HiG-Flow*.

Para o alcance do objetivo principal, definem-se os seguintes objetivos específicos:

- Desenvolver e treinar um modelo de rede neural que calcule os pesos de interpolação (como os gerados pelos Mínimos Quadrados Móveis - MLS), mantendo precisão numérica suficiente para não comprometer a convergência do método numérico
- Avaliar e otimizar a eficiência computacional do modelo desenvolvido, visando uma execução de inferência significativamente mais rápida do que o cálculo analítico matricial tradicional.
- Projetar uma metodologia de tratamento de dados que solucione o problema da dimensionalidade variável dos estênceis de malhas hierárquicas.
- Integrar a rede neural treinada nativamente à base de código em linguagem C do simulador *HiG-Flow*.

### 3 Plano de Trabalho e Cronograma

No primeiro trimestre, o aluno deve se familiarizar com o software *HiG-Flow* ([1], [2], [5], [6]), sendo capaz de realizar simulações e alterar o código quando pertinente. No segundo trimestre, deverá revisar técnicas de Machine Learning (tais como descritas em [7], [8], [9]), escolhendo qual tipo de rede neural deverá ser empregada, assim como a ferramenta utilizada.

A partir do segundo semestre, o aluno se dedicará à implementação computacional do projeto. Dessa forma, deve treinar e testar a rede neural, assim como portabilizá-la para o *HiG-Flow*.

### 4 Materiais e Métodos

Atualmente no *HiG-Flow*, para cada ponto de um estencil que não corresponde a nenhum ponto da malha, é necessário executar uma interpolação. Seja  $\mathbf{x}$  esse ponto fora da malha e suponha que queremos interpolar uma função  $u : \mathbb{R}^d \rightarrow \mathbb{R}$  pelas funções linearmente independentes e suaves  $\Phi_i : \mathbb{R}^d \rightarrow \mathbb{R}$ ,  $i = 1, \dots, n$  tal que a aproximação  $U(\mathbf{x})$  seja dada por:

$$U(\mathbf{x}) = \sum_{j=1}^n c_j \Phi_j(\mathbf{x}) \quad (1)$$

Dados  $m$  pontos da malha  $x_1, \dots, x_m$  com  $m > n$  e  $u_i = u(x_i)$ ,  $i = 1, \dots, m$ , definimos  $\mathbf{u} = (u_1, \dots, u_m)$ ,  $\mathbf{U} = (U(x_1), \dots, U(x_m))$ ,  $\mathbf{c} = (c_1, \dots, c_n)$  e a função de erro

$$E(\mathbf{c}) = \|\mathbf{U} - \mathbf{u}\|_2^2 = \sum_{i=1}^m (U(x_i) - u_i)^2 \lambda_i(\mathbf{x}) \quad (2)$$

que usa a norma induzida por um produto interno ponderado pelas funções de peso:

$$\lambda_i(\mathbf{x}) = \frac{1}{\|\mathbf{x} - x_i\|_2} \quad (3)$$

Definindo  $W \in \mathbb{R}^{m \times m}$  tal que  $W_{ij} = \delta_{ij} \sqrt{\lambda_i(\mathbf{x})}$  e  $P \in \mathbb{R}^{m \times n}$  com  $P_{ij} = \Phi_j(x_i)$ , obtemos o seguinte sistema de mínimos quadrados:

$$(WP)^\top WP \mathbf{c}(\mathbf{x}) = (WP)^\top W \mathbf{u} \quad (4)$$

Podemos fazer a decomposição QR da matriz  $WP$ , que resulta em

$$WP = Q \begin{bmatrix} R \\ 0 \end{bmatrix} = \begin{bmatrix} Q_{\parallel} & Q_{\perp} \end{bmatrix} \begin{bmatrix} R \\ 0 \end{bmatrix} \quad (5)$$

em que  $Q \in \mathbb{R}^{m \times m}$  é ortogonal,  $Q_{\parallel} \in \mathbb{R}^{m \times n}$ ,  $Q_{\perp} \in \mathbb{R}^{m \times (m-n)}$  são submatrizes de  $Q$  e  $R \in \mathbb{R}^{n \times n}$  é triangular superior. [1] diz que o vetor  $\mathbf{c}$  é dado por

$$\mathbf{c}(\mathbf{x}) = R^{-1} Q_{\parallel}^{\top} W \mathbf{u} \quad (6)$$

Como o objetivo é juntar essas aproximações com o método de diferenças finitas, devemos calcular os coeficientes de interpolação  $w_i$  que formam a combinação linear dos valores  $u_i$ . Logo, definindo  $\Phi = (\Phi_1(\mathbf{x}), \dots, \Phi_n(\mathbf{x}))$ , temos

$$U(\mathbf{x}) = \mathbf{c}^{\top} \Phi = \mathbf{u}^{\top} \underbrace{W Q_{\parallel} R^{-1}}_{\mathbf{w}} \Phi = \sum_{i=1}^m w_i(\mathbf{x}) u(x_i) \quad (7)$$

Esses pesos  $w_i$  podem então ser incorporados na matriz do sistema das diferenças finitas. De acordo com [1], o gargalo computacional se encontra principalmente no cálculo da decomposição QR.

## 4.1 Possíveis caminhos

Para mitigar esse custo, este projeto investigará, inicialmente, duas abordagens baseadas em Deep Learning:

- **MLP com Padding:** A arquitetura *Multilayer Perceptron* (MLP) possui uma estrutura simples e altamente tratada na literatura, facilitando a implementação dessa abordagem. Nas *HiG-Trees*, apesar das vizinhanças utilizadas nas interpolações possuírem quantidade variável de pontos, esse valor é limitado. Para lidar com isso, cria-se uma estrutura com o tamanho máximo e, se um esquema de interpolação conter menos pontos que o tamanho máximo, pode-se usar o *padding* nas entradas que sobraram. Essa técnica sinaliza para o modelo que os valores ali não devem ser considerados para fins de treino e inferência [7].
- **Deep Operator Networks (DeepONet):** Uma abordagem para o problema é a de que a rede deve aprender o operador de interpolação que mapeia coordenadas espaciais para pesos, indicando um possível uso da arquitetura DeepONet. Esta estrutura é composta por uma *branch net* (para codificar os sensores da malha) e uma *trunk net* (para as coordenadas de consulta), sendo eficaz no aprendizado de operadores lineares e não-lineares em física computacional [10].

Uma observação relevante é que, se  $U$  é um polinômio completo de grau  $K$  (como já implementado no *HiG-Flow*), ou seja, possui todos os monômios de grau até  $K$ , então os pesos  $w_i$  são invariantes por transformações rígidas (translações, rotações e reflexões) [1]. Com isso, cada esquema de interpolação pode ser centralizado na origem, diminuindo a complexidade do problema de aprendizado.

O treinamento será realizado utilizando um *dataset* gerado sinteticamente pelo solver determinístico do *HiG-Flow*. Para a implementação, poderão ser utilizadas bibliotecas como PyTorch ou TensorFlow para o treinamento dos modelos, bem como ferramentas de integração com código em C, como o *ONNX Runtime*, visando maximizar o desempenho durante o tempo de inferência.

## 5 Colaborações e parcerias

O trabalho será desenvolvido em parceria com o Laboratório de Matemática Aplicada e Computação Científica (LMACC), vinculado ao Instituto de Ciências Matemáticas e de Computação (ICMC) da USP São Carlos. Caso seja necessário um grande poder computacional, o LMACC concede acesso ao Cluster Euler, de alto desempenho.

## Referências

- [1] F. Sousa, C. Lages, J. Ansoni, A. Castelo e A. Simao, “A finite difference method with meshless interpolation for incompressible flows in non-graded tree-based grids,” *Journal of Computational Physics*, v. 396, pp. 848–866, 2019, ISSN: 0021-9991. DOI: 10.1016/j.jcp.2019.07.011 endereço: <https://www.sciencedirect.com/science/article/pii/S0021999119304966> (ver pp. 1–4).
- [2] A. T. G. da Silva, “Modelagem de interfaces em escoamentos bifásicos de fluidos viscoelásticos usando o software HiG-Flow,” Tese de Doutorado, Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo, São Carlos, 2025. DOI: 10.11606/T.55.2025.tde-07082025-150725 endereço: <https://www.teses.usp.br/teses/disponiveis/55/55134/tde-07082025-150725/pt-br.php> (ver pp. 2, 3).
- [3] J. J. Marziale, J. Sun, D. Salac e J. Chen, “Grid-agnostic volume of fluid approach with interface sharpening and surface tension for compressible multiphase flows,” *Computers & Fluids*, v. 301, p. 106794, 2025, ISSN: 0045-7930. DOI: <https://doi.org/10.1016/j.compfluid.2025.106794> endereço: <https://www.sciencedirect.com/science/article/pii/S0045793025002543> (ver p. 2).
- [4] J. Mencinger e I. Žun, “A PLIC–VOF method suited for adaptive moving grids,” *Journal of Computational Physics*, v. 230, n. 3, pp. 644–663, 2011, ISSN: 0021-9991. DOI: <https://doi.org/10.1016/j.jcp.2010.10.010> endereço: <https://www.sciencedirect.com/science/article/pii/S0021999110005553> (ver p. 2).

- [5] D. G. L. Raymundo, “Simulações numéricas de escoamentos bifásicos eletro-osmóticos viscoelásticos em malhas hierárquicas,” Dissertação de Mestrado, Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo, São Carlos, 2024. DOI: 10.11606/D.55.2024.tde-10012025-165209 endereço: <https://www.teses.usp.br/teses/disponiveis/55/55134/tde-10012025-165209/pt-br.php> (ver p. 3).
- [6] A. C. Filho e collaborators, *HiG-Flow*, <https://github.com/antoniocastelofilho/HigFlow>, GitHub repository, accessed May 10a, 2026 (ver p. 3).
- [7] I. Goodfellow, Y. Bengio e A. Courville, *Deep Learning*. MIT Press, 2016, <http://www.deeplearningbook.org> (ver pp. 3, 4).
- [8] A. d. P. Braga, T. B. Ludermir e A. C. P. d. L. F. d. Carvalho, *Redes neurais artificiais: teoria e aplicações*. LTC, 2007 (ver p. 3).
- [9] S. Haykin, *Neural Networks: A Comprehensive Foundation*, 2<sup>a</sup> ed. Upper Saddle River, NJ: Prentice Hall, 1999, ISBN: 9780132733502 (ver p. 3).
- [10] L. Lu, P. Jin, G. Pang, Z. Zhang e G. E. Karniadakis, “Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators,” *Nature Machine Intelligence*, v. 3, n. 3, pp. 218–229, 2021. DOI: 10.1038/s42256-021-00302-5 (ver p. 4).